



Escuela Politécnica Nacional

Facultad de Ingeniería de Sistemas

**Construcción y Evolución de
Software - GR2SW**

EXAMEN PRÁCTICO

MANTENIMIENTO DE SOFTWARE

“Sistema de Gestión de Cafetería”



Sistema de Gestión de Cafetería

NOMBRE:

Josué Sebastián Cuenca Cuenca

Profesor: MSc. Egüez Sarzosa Vicente Adrián

2026 - A



Contenido

1. Introducción	4
2. Objetivo General.....	4
3. Objetivos Específicos	4
4. Descripción General del Proyecto	5
4.1 Frontend	5
4.2 Backend	5
5. Tecnologías Utilizadas	6
6. Diagnóstico Inicial del CRUD	6
6.1 Problemas Identificados	6
7. Arquitectura del Sistema	7
7.1 Módulo Frontend	7
7.2 Módulo Backend	7
8. Mantenimiento Correctivo	8
8.1 Problemas Corregidos	8
9. Mantenimiento Adaptativo	9
9.1 Variables de Entorno	9
9.2 Seguridad mediante API-Key	9
10. Mantenimiento Perfectivo	10
10.1 Implementación de búsqueda dinámica.....	10
10.2 Implementación de edición de productos	10
10.3 Eliminación con confirmación	10
10.4 Integración con Event Manager	10
10.5 Mejora visual	10
10.6 Organización del código	11
11. Mantenimiento Preventivo.....	11
11.1 Sanitización Básica	11
11.2 Validaciones Preventivas.....	11
11.3 Manejo de Errores.....	11
11.4 Prevención de Eliminaciones Accidentales	12



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA EN SOFTWARE

12. Registro de Eventos y Auditoría	12
12.1 Información registrada.....	12
12.2 Eventos soportados	13
12.3 Beneficios de la auditoría.....	13
13. Seguridad Implementada	13
13.1 API-Key	13
13.2 Sanitización	13
13.3 Validaciones	13
14. Pruebas Realizadas	13
14.1 Pruebas CRUD	14
14.2 Validaciones	14
14.3 Integración Backend.....	14
14.4 Pruebas Básicas con Jest.....	14
15. Evidencias del Sistema de Gestión de Cafetería	14
.....	20
16. Clasificación de los Cambios según el Tipo de Mantenimiento	21
17. Cumplimiento de la Rúbrica	21
18. Resultados Obtenidos	21
19. Conclusiones	22



1. Introducción

El mantenimiento de software constituye una de las actividades más importantes dentro del ciclo de vida de un sistema informático, ya que permite corregir errores, adaptar aplicaciones a nuevos entornos, mejorar funcionalidades y prevenir fallos futuros.

En el presente trabajo se desarrolló un proceso completo de mantenimiento sobre un sistema CRUD denominado **Sistema de Gestión de Cafetería**, cuyo propósito es administrar productos de cafetería mediante operaciones de creación, consulta, actualización y eliminación de registros.

El sistema tiene como objetivo evolucionar un CRUD básico hacia una aplicación más robusta, segura, trazable y mantenible, incorporando conceptos reales de mantenimiento de software, pruebas, auditoría, seguridad y documentación técnica.

Durante el desarrollo se aplicaron los cuatro tipos principales de mantenimiento de software:

- Mantenimiento Correctivo.
- Mantenimiento Adaptativo.
- Mantenimiento Perfectivo.
- Mantenimiento Preventivo.

Además, se incorporaron mecanismos de validación, sanitización, trazabilidad mediante eventos, seguridad con API-Key, configuración mediante variables de entorno y documentación técnica estructurada.

2. Objetivo General

Aplicar técnicas de mantenimiento de software al Sistema de Gestión de Cafetería con el fin de mejorar su calidad, seguridad, mantenibilidad, estabilidad y trazabilidad, cumpliendo con los requerimientos establecidos en el examen práctico.

3. Objetivos Específicos

- Analizar las debilidades técnicas del CRUD inicial.
- Implementar validaciones y correcciones funcionales.
- Adaptar el sistema a mecanismos de seguridad mediante API-Key.
- Mejorar la mantenibilidad y organización del proyecto.
- Incorporar registros de auditoría mediante eventos.
- Aplicar sanitización básica para reducir riesgos de seguridad.



- Implementar pruebas funcionales y validaciones.
- Documentar técnicamente el funcionamiento del sistema.

4. Descripción General del Proyecto

El proyecto desarrollado corresponde a un sistema CRUD para la gestión de productos de cafetería. El sistema permite registrar productos, consultarlos, modificarlos, eliminarlos y realizar búsquedas dinámicas.

La arquitectura del sistema está dividida en dos componentes principales:

4.1 Frontend

El frontend fue desarrollado utilizando:

- HTML5
- CSS3
- JavaScript

La interfaz permite:

- Registrar productos.
- Buscar productos.
- Editar registros.
- Eliminar productos.
- Visualizar productos almacenados.

Los productos administrados contienen información como:

- Nombre.
- Categoría.
- Precio.
- Stock.

4.2 Backend

El backend fue desarrollado utilizando:

- NestJS
- TypeScript
- SQLite



- TypeORM

El backend se encarga de:

- Registrar eventos del sistema.
- Almacenar auditorías.
- Validar solicitudes.
- Gestionar endpoints REST.
- Proteger endpoints mediante API-Key.

5. Tecnologías Utilizadas

Tecnología	Uso
HTML5	Estructura del frontend
CSS3	Diseño visual
JavaScript	Lógica CRUD
NestJS	Backend
TypeScript	Programación backend
SQLite	Base de datos
TypeORM	Persistencia
Jest	Pruebas básicas
API-Key	Seguridad
dotenv	Variables de entorno

6. Diagnóstico Inicial del CRUD

Antes de aplicar mantenimiento, el sistema presentaba características propias de un CRUD básico y múltiples debilidades técnicas.

6.1 Problemas Identificados

Se detectaron los siguientes problemas:

- Ausencia de validaciones completas.
- Posibilidad de ingresar datos inválidos.



- Falta de control sobre precios negativos.
- Posibilidad de registrar stock negativo.
- Ausencia de trazabilidad.
- Falta de auditoría de operaciones.
- Ausencia de mecanismos de seguridad.
- Falta de sanitización básica.
- Falta de documentación estructurada.
- Baja mantenibilidad del código.

Estas debilidades representan deuda técnica, ya que incrementan el riesgo de errores futuros y dificultan la evolución del sistema.

7. Arquitectura del Sistema

El sistema fue organizado en dos módulos principales:

7.1 Módulo Frontend

Contiene:

- index.html
- style.css
- app.js
- Carpeta img

Funciones principales:

- Gestión visual del CRUD.
- Formularios.
- Validaciones.
- Búsquedas.
- Comunicación con el backend.

7.2 Módulo Backend

Contiene:

- Módulo events



- Guardias de seguridad
- Configuración de base de datos
- Controladores REST
- Servicios de persistencia

Endpoints principales:

POST /events

GET /events

GET /events/source/:source

GET /events/entity/:entity

GET /health

GET /stats

8. Mantenimiento Correctivo

El mantenimiento correctivo se aplicó para solucionar errores funcionales presentes en el sistema inicial.

8.1 Problemas Corregidos

Validación de nombre vacío

Anteriormente el sistema permitía registrar productos sin nombre.

Se implementó:

```
if(nombre.trim() === ""){  
    alert("El nombre es obligatorio");  
    return;  
}
```

Validación de categoría vacía

Se evitó el registro de categorías vacías.

```
if(categoria.trim() === ""){  
    alert("La categoría es obligatoria");  
    return;  
}
```




Validación de precio

Se corrigió el problema de precios negativos o inválidos.

```
if(isNaN(precio) || precio <= 0){  
    alert("El precio debe ser mayor a 0");  
    return;  
}
```

Validación de stock

Se evitó registrar stock negativo.

```
if(isNaN(stock) || stock < 0){  
    alert("El stock no puede ser negativo");  
    return;  
}
```

Manejo de precios decimales

En el formulario se agregó:

```
<input type="number" step="0.01" id="precio" placeholder="Precio" required>
```

Esto permite manejar correctamente valores monetarios.

9. Mantenimiento Adaptativo

El mantenimiento adaptativo permitió que el sistema pueda operar bajo nuevas restricciones y configuraciones.

9.1 Variables de Entorno

Se implementó un archivo *.env* para separar configuraciones sensibles.

Ejemplo:

PORT = 3000

API_KEY = FIS-EPN-2026

Esto mejora la flexibilidad y facilita cambios futuros de infraestructura.

9.2 Seguridad mediante API-Key

Se implementó protección para los endpoints del backend mediante una *API-Key* personalizada.



Cabecera utilizada:

x-epn-key: FIS-EPN-2026

Cada solicitud enviada desde el frontend incorpora automáticamente esta cabecera.

Esto permite:

- Restringir accesos.
- Validar clientes autorizados.
- Mejorar la seguridad del sistema.

10. Mantenimiento Perfectivo

El mantenimiento perfectivo tuvo como objetivo mejorar la funcionalidad, claridad y mantenibilidad del sistema.

10.1 Implementación de búsqueda dinámica

Se agregó un sistema de búsqueda en tiempo real para encontrar productos rápidamente.

10.2 Implementación de edición de productos

Se agregó la capacidad de actualizar registros existentes.

10.3 Eliminación con confirmación

Antes de eliminar un producto el sistema solicita confirmación al usuario.

Esto reduce eliminaciones accidentales.

10.4 Integración con Event Manager

El frontend fue integrado con el backend para registrar eventos automáticamente.

Eventos registrados:

- CREATE
- UPDATE
- DELETE
- QUERY

10.5 Mejora visual

Se incorporó:

- Diseño más limpio.
- Logo personalizado.



- Organización visual del CRUD.

10.6 Organización del código

Se mejoró la separación lógica del sistema:

- Funciones CRUD.
- Eventos.
- Validaciones.
- Renderizado.
- Comunicación REST.

Esto facilita el mantenimiento futuro.

11. Mantenimiento Preventivo

El mantenimiento preventivo fue aplicado para reducir riesgos futuros y prevenir vulnerabilidades.

11.1 Sanitización Básica

Se implementó sanitización básica para evitar etiquetas HTML peligrosas.

```
.replace(/</g, "&lt;")  
.replace(/>/g, "&gt;");
```

Esto reduce riesgos básicos relacionados con XSS.

11.2 Validaciones Preventivas

Se agregaron validaciones antes de procesar cualquier dato ingresado.

11.3 Manejo de Errores

Se implementaron bloques try-catch para capturar errores durante el envío de eventos.



```
try{  
  await fetch("http://localhost:3000/events", {  
    method: "POST",  
    headers: {  
      "Content-Type": "application/json",  
      "x-epn-key": "FIS-EPN-2026"  
    },  
    body: JSON.stringify({  
      source: "cafeteria-crud",  
      entity: "producto",  
      action: type.toUpperCase(),  
      title: "Evento de cafetería",  
      description: `Acción ${type} realizada en el sistema de cafetería`,  
      payload: data  
    })  
  });  
  console.log(`[LOG INFO] Evento ${type.toUpperCase()} enviado correctamente`);  
}catch(error){  
  console.log(error);  
  console.error(`[LOG ERROR] Falló el envío del evento ${type.toUpperCase()}`);  
}
```

11.4 Prevención de Eliminaciones Accidentales

El sistema solicita confirmación antes de eliminar registros.

12. Registro de Eventos y Auditoría

Uno de los componentes más importantes del sistema es el registro de auditoría.

Cada operación importante genera un evento estructurado que es almacenado en el backend.

12.1 Información registrada

Cada evento almacena:

- Tipo de acción.
- Entidad afectada.



- Origen del evento.
- Fecha.
- Descripción.
- Payload asociado.

12.2 Eventos soportados

Evento	Descripción
CREATE	Registro de nuevos productos
UPDATE	Modificación de productos
DELETE	Eliminación de productos
QUERY	Búsquedas realizadas

12.3 Beneficios de la auditoría

- Trazabilidad.
- Seguimiento de acciones.
- Control de cambios.
- Depuración de errores.
- Monitoreo del sistema.

13. Seguridad Implementada

El sistema incorpora mecanismos básicos de seguridad.

13.1 API-Key

Protección de endpoints mediante cabeceras personalizadas.

13.2 Sanitización

Prevención básica de inyección HTML.

13.3 Validaciones

Validación de tipos y restricciones de datos.

14. Pruebas Realizadas

Se realizaron pruebas funcionales y validaciones manuales.



14.1 Pruebas CRUD

Se verificó:

- Registro correcto.
- Edición correcta.
- Eliminación correcta.
- Búsqueda correcta.

14.2 Validaciones

Se comprobó:

- Nombre obligatorio.
- Precio válido.
- Stock válido.
- Categoría obligatoria.

14.3 Integración Backend

Se verificó:

- Registro de eventos.
- Recepción de payloads.
- Consulta de eventos.
- Protección mediante API-Key.

14.4 Pruebas Básicas con Jest

El sistema incorpora pruebas básicas para validar operaciones simples y comportamiento esperado.

15. Evidencias del Sistema de Gestión de Cafetería

1. Interfaz principal.



The screenshot displays the main interface of the 'Sistema de Gestión de Cafetería'. At the top center is a logo for 'Sistema de Gestión de Cafetería'. Below the logo, the title 'Sistema de Gestión de Cafetería' is centered. The main form area contains four input fields: 'Nombre del producto', 'Categoría', 'Precio', and 'Stock'. Below these fields is a prominent brown button labeled 'Guardar Producto'. At the bottom left, under the heading 'Productos', there is a search input field with the placeholder text 'Buscar producto...'.



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA EN SOFTWARE

2. Registro de producto.



Sistema de Gestión de Cafetería

Productos

Cafe Negro

Categoría: Bebida

Precio: \$2.5

Stock: 3

3. Edición de producto.



Sistema de Gestión de Cafetería

Productos

Mocaccino

Categoría: Bebida

Precio: \$4.5

Stock: 4



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA EN SOFTWARE

4. Eliminación.

Sistema de Gestión de Cafetería

Confirme la acción en la página localhost

¿Seguro que desea eliminar este producto?

Productos

Mocaccino

Categoría: Bebida

Precio: \$4.5

Stock: 4

5. Búsqueda.

Productos

Te de valeriana

Categoría: Bebida

Precio: \$2.8

Stock: 2

Te de manzanilla

Categoría: Bebida

Precio: \$3.5

Stock: 6

Cafe Negro

Categoría: Bebida

Precio: \$3.5

Stock: 4

Productos

Cafe Negro

Categoría: Bebida

Precio: \$3.5

Stock: 4



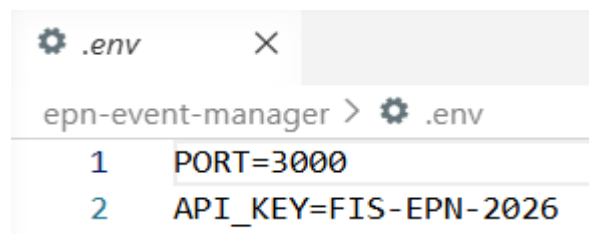
6. Validaciones.

```
if(nombre.trim() === ""){  
    alert("El nombre es obligatorio");  
    return;  
}  
  
if(categoria.trim() === ""){  
    alert("La categoría es obligatoria");  
    return;  
}  
  
if(isNaN(precio) || precio <= 0){  
    alert("El precio debe ser mayor a 0");  
    return;  
}  
  
if(isNaN(stock) || stock < 0){  
    alert("El stock no puede ser negativo");  
    return;  
}
```

7. Código de sanitización.

```
const safeNombre = nombre  
    .replace(/</g, "&lt;")  
    .replace(/>/g, "&gt;");  
  
const safeCategoria = categoria  
    .replace(/</g, "&lt;")  
    .replace(/>/g, "&gt;");
```

8. Archivo .env.



9. Logs de eventos.

```
[Nest] 6040 - 30/05/2026, 17:59:05 LOG [EventsService] Evento CREATE registrado correctamente  
[Nest] 6040 - 30/05/2026, 17:59:56 LOG [EventsService] Evento DELETE registrado correctamente  
[Nest] 6040 - 30/05/2026, 18:01:02 LOG [EventsService] Evento CREATE registrado correctamente  
[Nest] 6040 - 30/05/2026, 18:01:53 LOG [EventsService] Evento UPDATE registrado correctamente  
[Nest] 6040 - 30/05/2026, 18:03:02 LOG [EventsService] Evento DELETE registrado correctamente  
[Nest] 6040 - 30/05/2026, 18:03:34 LOG [EventsService] Evento CREATE registrado correctamente  
[Nest] 6040 - 30/05/2026, 18:03:43 LOG [EventsService] Evento CREATE registrado correctamente  
[Nest] 6040 - 30/05/2026, 18:03:56 LOG [EventsService] Evento CREATE registrado correctamente  
[Nest] 6040 - 30/05/2026, 18:04:48 LOG [EventsService] Evento QUERY registrado correctamente  
□
```

[illegible]

localhost:3000 localhost:3000/health

Dar formato al texto ☐

```
{"status": "ok", "timestamp": "30/5/2026, 18:13:15"}
```

localhost:3000 localhost:3000/stats

Dar formato al texto ☐

```
{"create":38,"update":18,"delete":11,"total":67}
```



Sistema de Gestión de Cafetería

Productos

Cafe Negro

Categoría: Bebida

Precio: \$2.8

Stock: 2



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA EN SOFTWARE



Sistema de Gestión de Cafetería

Por favor, rellene este campo.

Guardar Producto



Notificación de la página localhost

El precio debe ser mayor a 0

Cerrar

Guardar Producto

Productos



Sistema de Gestión de Cafetería

Actualizar Producto

Productos

Te de manzanilla

Categoría: Bebida

Precio: \$3.5

Stock: 6

Editar

Eliminar



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA DE SISTEMAS
INGENIERÍA EN SOFTWARE



Confirme la acción en la página localhost

¿Seguro que deseas eliminar este producto?

Sí

No

Te de valeriana

Bebida

3,5

6

Actualizar Producto

Productos

Buscar producto...

Te de manzanilla

Categoría: Bebida

Precio: \$3.5

Stock: 6

Editar

Eliminar

Productos

Buscar producto...

Cafe Negro

Categoría: Bebida

Precio: \$3.5

Stock: 6

Editar

Eliminar

Mocaccino

Categoría: Bebida

Precio: \$1.5

Stock: 3

Editar

Eliminar

Te de valeriana

Categoría: Bebida

Precio: \$2.5

Stock: 2

Editar

Eliminar

Productos

Mocaccino

Mocaccino

Categoría: Bebida

Precio: \$1.5

Stock: 3

Editar

Eliminar



16. Clasificación de los Cambios según el Tipo de Mantenimiento

Tipo	Cambios Aplicados
Correctivo	Corrección de validaciones y errores
Adaptativo	API-Key y variables de entorno
Perfectivo	Búsqueda, edición, eventos, UI
Preventivo	Sanitización y manejo de errores

17. Cumplimiento de la Rúbrica

Diagnóstico

Se identificaron problemas funcionales, de seguridad y mantenibilidad.

Intervención Técnica

Se realizaron mejoras completas tanto en frontend como backend.

Clasificación y Teoría

Los cambios fueron clasificados formalmente según la taxonomía de mantenimiento de software.

Prueba de Vida

El sistema demuestra funcionamiento correcto en tiempo real mediante operaciones CRUD y registro de eventos.

18. Resultados Obtenidos

Después del mantenimiento aplicado, el sistema logró:

- Mayor estabilidad.
- Mejor seguridad.
- Mejor experiencia de usuario.
- Validaciones más robustas.
- Trazabilidad completa.
- Mayor mantenibilidad.
- Mejor organización del código.
- Integración backend funcional.



19. Conclusiones

El **Sistema de Gestión de Cafetería** evolucionó desde un **CRUD** básico hacia una aplicación más robusta y mantenible mediante la aplicación de técnicas reales de mantenimiento de software.

Se aplicaron correctamente los cuatro tipos principales de mantenimiento:

- Correctivo.
- Adaptativo.
- Perfectivo.
- Preventivo.

Además, se incorporaron mecanismos de validación, auditoría, seguridad y organización del código que mejoran significativamente la calidad del sistema.

El sistema cumple con los requerimientos planteados en el examen práctico, demostrando una evolución técnica real del software y una mejora importante en seguridad, estabilidad y mantenibilidad.